

Event Venue Finder for Münster

A Web-Based GIS Application for Spatial Data Infrastructures

Khanh Giang Le 556087

Lerissa Merril Menezes 564864

This document includes language and structure refinements assisted
by an AI language model

CONTENTS

1.	Introduction	3
2.	System Architecture	3
3	Datasets and Data Preparation	3
4.	Frontend Implementation	4
5.	Challenges and Learnings	5

1. Introduction

This report describes the design, architecture, and implementation of the Event Venue Finder for Münster, a web-based GIS application developed for the Spatial Data Infrastructures course. The application provides an interactive map for discovering event venues across Münster, integrating a self-compiled venue dataset with public bus stop locations. Both datasets are exposed through OGC-compliant web services to a Leaflet.js frontend, allowing users to explore venue attributes and assess public transport accessibility in a single interface.

2. System Architecture

The system follows a three-tier architecture consisting of a data tier, a service tier, and a presentation tier. At the data tier, both shapefiles are stored within GeoServer's data directory. At the service tier, GeoServer acts as the geospatial middleware, publishing each dataset as a distinct store within a workspace and exposing it via OGC-compliant Web Feature Service (WFS) endpoints. The frontend consumes these endpoints at the presentation tier using JavaScript's Fetch API, requesting data in GeoJSON format via the `outputFormat=application/json` parameter.

GeoServer was configured with Cross-Origin Resource Sharing (CORS) support to allow the locally hosted frontend to make requests to the GeoServer instance running on a different port. This required editing the `web.xml` configuration file within GeoServer's deployment directory to enable the `CrossOriginFilter`, specifying wildcard allowances for origins, methods, and headers. The filter mapping was applied to all URL patterns to ensure both WFS and WMS endpoints were covered.

The frontend is structured as three separate files, `index.html`, `style.css`, and `map`. The HTML file provides structure and links to dependencies. The CSS file handles all visual styling including popup appearance, the map legend, header, and tooltip design. The JavaScript file contains all application logic.

3. Dataset and Data Preparation

Two datasets underpin the application. The first is a self-compiled GeoJSON file of event venues in Münster, with each feature carrying over twenty attributes including venue type, indoor or outdoor category, address, minimum and maximum capacity, availability on

weekends and evenings, whether alcohol and food are permitted, noise restriction level, permit requirements, contact information, suitable event types, and general notes. The second dataset is an external GeoJSON file of Münster bus stops, used to support public transport proximity analysis.

Since GeoServer does not natively support GeoJSON as a data store without additional plugins, both files were converted to ESRI Shapefile format using QGIS prior to publication. This conversion introduced several limitations that required careful handling downstream. Field names were truncated to ten characters, meaning attribute references in the frontend had to match the shortened names returned by GeoServer. Array-type properties such as `suitable_for` were flattened into comma-separated strings and had to be parsed back on the client side. Encoding was set explicitly to UTF-8 during export, and a .cpg companion file was added to ensure German umlauts rendered correctly.

4. Frontend Implementation

The mapping interface is built on Leaflet.js, an open-source JavaScript library for interactive maps. On page load, both WFS endpoints are fetched simultaneously using `Promise.all()`, ensuring neither layer renders before both datasets are fully available. This is particularly important for the nearest bus stop calculation, which requires the complete bus stop dataset to be in memory.

Nearest bus stop proximity is calculated entirely client-side using the Haversine formula, which computes great-circle distance between two geographic coordinates. For each venue, the function iterates through all bus stop features and identifies the one with the minimum distance. The result including the stop name and distance in metres is displayed at the top of each venue's popup. This approach avoids the need for a server-side spatial query and keeps the architecture simple while remaining accurate for city-scale distances.

Walking route generation is implemented using Leaflet Routing Machine, which interfaces with the OSRM pedestrian routing service. When a user clicks a venue marker, a route is drawn from the venue to its nearest bus stop. A global variable tracks the active route control so that clicking a different venue removes the previous route before drawing a new one. Waypoint markers are suppressed to avoid visual clutter, and the turn-by-turn directions panel is hidden via CSS, leaving only the route line visible on the map.

Venue popups are dynamically generated from GeoServer response properties and include null handling for optional fields such as contact phone, email, and website, so that missing values do not render empty icons or broken layout elements. Boolean attributes such as alcohol permitted, restrooms available, and food allowed are rendered as colour-coded

badges. The `suitable_for` field, which arrives as a comma-separated string due to Shapefile's inability to store arrays, is split and rendered as individual tags within the popup.

5. Challenges and Lessons Learned

Several technical challenges arose during implementation. The most persistent was CORS configuration in GeoServer. Enabling it required both a filter definition and a corresponding filter-mapping block in `web.xml`, and on Jetty 10 the `chainPreflight` parameter had to be set to false for preflight OPTIONS requests to be handled correctly, as without it, the browser continued to block requests despite the filter being present. Shapefile's structural constraints also introduced limitations such as truncated field names required updating all attribute references in the frontend, array values had to be reconstructed from strings, and encoding issues with German characters took multiple steps to resolve including re-exporting from QGIS and adding a `.cpg` file.

Working through these issues reinforced the value of understanding the full data pipeline from source format through geospatial middleware to the consuming application. It also highlighted the trade-offs inherent in format choices: while Shapefile is universally supported and easy to publish in GeoServer, its structural limitations make PostGIS a more appropriate choice for production systems with richer attribute schemas. Overall, the project provided practical experience with spatial data infrastructures, OGC service standards, client-side spatial computation, and the integration of open geospatial tools into a functioning web application.